

B1, BCRYPT Y AES

**Nombre: Albert Vallés,
David Ribera y
Mariona Pan**

Curso: ASIX2

ÍNDICE

Funciones de Hash Parte I	3
1. Funcionamiento y Tipos de Hashes	3
2. ¿Cómo se generan los hashes?	5
3. ¿Qué es una colisión?	8
4. ¿Para qué se usa un Hash?	11
5. ¿Cómo se usan los hashes en criptomonedas?	16
6. ¿Qué es una Rainbow Table?	22
7. Esquemas	25
8. Videos	28

Funciones de Hash Parte I

1. Funcionamiento y Tipos de Hashes

Una función de hash es un algoritmo matemático que toma una entrada de tamaño variable y la transforma en una salida de longitud fija. La salida de esta función se conoce como valor hash, digest o huella digital, y se utiliza para representar de manera única el contenido original. El proceso es unidireccional, lo que significa que, en teoría, es casi imposible obtener la entrada original a partir del valor hash.

¿Cómo Funciona una Función de Hash?

El funcionamiento de una función de hash se puede describir en términos generales de la siguiente manera:

1. **Entrada de Datos:** Cualquier tipo de dato puede ser la entrada: un texto, un archivo, una contraseña, una imagen, etc. La función de hash toma esta entrada, independientemente de su tamaño o formato.
2. **Procesamiento:** La función de hash procesa la entrada a través de una serie de operaciones matemáticas y lógicas. Estas operaciones pueden involucrar sumas, desplazamientos de bits, operaciones bit a bit, etc. El objetivo es transformar los datos de manera que incluso un pequeño cambio en la entrada produzca un cambio significativo en el valor de salida.
3. **Salida de Longitud Fija:** La salida, o el valor hash, tendrá una longitud fija, que no depende del tamaño de la entrada. Por ejemplo, si usas SHA-256 como algoritmo de hash, siempre obtendrás un valor de 256 bits (64 caracteres hexadecimales), sin importar si la entrada es un archivo de 1 KB o de 1 GB.
4. **Irreversibilidad:** La propiedad clave de las funciones de hash es que son irreversibles. Esto significa que, una vez que se genera un valor hash, es imposible (o prácticamente imposible) revertir el proceso y recuperar la entrada original solo con el valor hash.
5. **Determinismo:** Si se vuelve a procesar la misma entrada con el mismo algoritmo de hash, siempre se obtiene el **mismo valor hash**.
6. **Sensibilidad a Cambios Pequeños:** Si se cambia incluso un solo bit en la entrada, el valor hash generado será completamente diferente. Esta propiedad se conoce como el efecto avalancha, y es esencial para garantizar que cada entrada tenga un

hash único.

Tipos de Funciones de Hash

Las funciones de hash se pueden dividir en dos categorías principales: criptográficas y no criptográficas. Cada tipo tiene diferentes características y se utiliza en diferentes contextos.

1. Funciones de Hash Criptográficas (Seguras)

Las funciones de hash criptográficas están diseñadas para ser seguras y resistentes a manipulaciones, lo que significa que son difíciles de predecir, revertir o modificar. Son fundamentales en seguridad informática y en aplicaciones como la protección de contraseñas, firma digital y blockchain. La principal característica de estas funciones es que tienen propiedades que las hacen a prueba de colisiones, es decir, es extremadamente difícil que dos entradas diferentes generen el mismo hash (colisión).

Propiedades clave:

- **Resistencia a las colisiones:** Es casi imposible que dos entradas diferentes generen el mismo hash.
- **Resistencia a preimagen:** Dado un hash, es casi imposible encontrar la entrada que lo generó.
- **Resistencia a la segunda preimagen:** Dado un conjunto de entrada y su hash, es muy difícil encontrar una entrada diferente que tenga el mismo hash.
- **Determinismo:** La misma entrada siempre produce el mismo hash.

2. Funciones de Hash No Criptográficas

Las funciones de hash no criptográficas están diseñadas para ser rápidas y eficientes en términos de velocidad de ejecución. Sin embargo, no están diseñadas para ser seguras. A diferencia de las funciones criptográficas, las funciones de hash no criptográficas no se usan para proteger datos sensibles ni son resistentes a manipulaciones o colisiones. En cambio, se usan principalmente en aplicaciones que necesitan una forma rápida y eficiente de almacenar y recuperar datos.

Propiedades clave:

- **Velocidad:** Están optimizadas para ser muy rápidas en su ejecución, lo cual las hace útiles en aplicaciones de rendimiento.
- **No seguras:** No están diseñadas para resistir ataques. En consecuencia, no son adecuadas para proteger contraseñas o datos sensibles.

- **Sin necesidad de resistencia a las colisiones:** Dado que no están diseñadas para la seguridad, no se les requiere que eviten las colisiones de manera estricta.

2. ¿Cómo se generan los hashes?

Generar un hash es un proceso fundamental en informática, especialmente en criptografía. El propósito principal de un hash es tomar un conjunto de datos (que puede ser de cualquier tamaño) y transformarlo en una cadena de longitud fija, normalmente de forma irreversible, que representa de manera única esa entrada. El proceso de generar un hash sigue ciertos pasos y utiliza algoritmos específicos, como SHA-256.

Pasos para generar un hash

1. **Entrada (Input):** El proceso comienza con una entrada, que puede ser cualquier dato: texto, archivo, imagen, contraseña, etc. El tamaño de la entrada puede variar, desde una palabra pequeña como "Hola" hasta un archivo gigante como una película.

Ejemplo de entrada:

- "Hola Mundo"
 - "Contraseña123"
 - "Imagen.jpg"
2. **Aplicación del Algoritmo de Hash:** Una vez que se tiene la entrada, se le aplica un algoritmo de hash. El algoritmo es una función matemática que toma el dato de entrada y lo procesa. El tipo de algoritmo determina cómo se procesan los datos y qué características tendrá el hash resultante.

Hay muchos algoritmos de hash, cada uno con diferentes propiedades:

- **SHA-256:** Produce un hash de 256 bits. Es muy popular y utilizado en criptomonedas como Bitcoin.
 - **MD5:** Produce un hash de 128 bits. Aunque es rápido, no se recomienda para usos de seguridad debido a vulnerabilidades.
 - **SHA-1:** Produce un hash de 160 bits, pero también tiene vulnerabilidades.
3. El algoritmo de hash es diseñado para que:
 - La salida sea de longitud fija.
 - El mismo conjunto de datos de entrada siempre produzca el mismo hash.

- Sea casi imposible revertir el hash para obtener la entrada original (esto se llama irrevocabilidad).
- Sea sensible a cambios mínimos: incluso si solo cambias un carácter en la entrada, el hash resultante cambia completamente.

Salida (Hash): El resultado de aplicar el algoritmo es el hash, que es una cadena de texto, pero es muy diferente a la entrada original. Es de longitud fija y tiene características especiales que lo hacen único para cada entrada.

Por ejemplo, si usas SHA-256 para el texto "Hola Mundo", el hash generado será:

Entrada: "Hola Mundo"

Salida (hash):

a830d7beb04eb7549ce990fb7dc962e499a27230ee... (truncado)

4. El hash generado es una cadena hexadecimal de longitud fija (en este caso, 64 caracteres, ya que SHA-256 produce un hash de 256 bits).

Propiedades de los hashes

Para ser útil en seguridad, un buen algoritmo de hash debe tener ciertas características:

- **Determinismo:** El mismo dato de entrada siempre producirá el mismo hash.
- **Longitud fija:** El hash generado siempre tendrá la misma longitud, independientemente del tamaño de la entrada. Por ejemplo, SHA-256 siempre produce un hash de 256 bits, sin importar si el archivo de entrada tiene 10 MB o 1 KB.
- **Irreversibilidad:** Dado un hash, no es posible revertir el proceso para obtener la entrada original. Esto asegura que, si alguien ve el hash, no puede obtener el contenido original de manera sencilla.
- **Sensibilidad a pequeñas variaciones:** Si la entrada cambia en el mínimo detalle, el hash también cambiará radicalmente. Por ejemplo, si cambias "Hola Mundo" por "Hola mundo" (solo el caso de la "M"), el hash resultante será completamente diferente.
- **Resistencia a colisiones:** Es muy difícil que dos entradas diferentes generen el mismo hash. Si dos entradas diferentes generan el mismo hash, se dice que hay una **colisión**, y esto sería un problema grave en la seguridad de un algoritmo de hash. Buenas funciones de hash (como SHA-256) están diseñadas para minimizar las colisiones.

Ejemplo de cómo se genera un hash con SHA-256

Vamos a ver un ejemplo paso a paso utilizando el algoritmo SHA-256 para generar el hash de la entrada "Hola Mundo".

1. **Entrada:** "Hola Mundo"
2. **Aplicación del algoritmo SHA-256:** SHA-256 procesará "Hola Mundo" utilizando una serie de operaciones matemáticas. La entrada se divide en bloques y se aplica la función de hash repetidamente.

Resultado: Después de procesar, el algoritmo generará una salida de 256 bits (64 caracteres en formato hexadecimal).

El hash resultante para "Hola Mundo" (es solo un ejemplo, la salida completa sería mucho más larga) sería algo como:

a830d7beb04eb7549ce990fb7dc962e499a27230ee437be1535be0c857f7ee40

Aplicaciones comunes de los hashes:

Los hashes se utilizan en una variedad de aplicaciones en tecnología, especialmente en seguridad informática:

1. **Contraseñas:** En lugar de almacenar contraseñas de usuarios, los sistemas guardan los hashes de las contraseñas. Cuando un usuario ingresa su contraseña, el sistema la hashea y compara el hash con el que tiene almacenado.
2. **Integridad de archivos:** Los hashes permiten verificar que un archivo no ha sido alterado. Al descargar un archivo de internet, el sitio proporciona un hash del archivo. Luego, tú puedes calcular el hash del archivo descargado y compararlo con el proporcionado. Si los hashes coinciden, el archivo no fue alterado.
3. **Criptomonedas:** En Bitcoin y otras criptomonedas, los bloques de transacciones están vinculados por sus hashes, y los mineros usan hashes para asegurar la red y encontrar nuevos bloques.

3. ¿Qué es una colisión?

Cuando dos entradas diferentes generan el mismo valor hash. Las buenas funciones de hash criptográficas minimizan esta posibilidad.

Una colisión es mala, porque el hash debería ser único como una huella digital.

Ejemplo:

- Metes “archivo1” → da abc123
- Metes “archivo2” → también da abc123

Esto significaría que podrías engañar un sistema si logras encontrar una colisión.

Lo que supone un problema de seguridad en funciones de hash débiles, como MD5, que ya no se consideran seguras.

Soluciones para Prevenir Colisiones

Para prevenir los riesgos de colisiones, las funciones de hash modernas, como SHA-256 y SHA-3, están diseñadas con propiedades criptográficas más robustas, lo que dificulta enormemente encontrar colisiones. Además, para proteger aún más los sistemas, se pueden emplear técnicas como:

- **Salting:** La adición de un valor aleatorio (sal) a los datos antes de aplicar la función de hash puede hacer que los hashes de entradas iguales sean completamente diferentes, incluso si las entradas son idénticas. Esto es común en el almacenamiento de contraseñas.
- **Uso de funciones de hash seguras y actualizadas:** Siempre que sea posible, es importante utilizar funciones de hash modernas y probadas, como SHA-256 o SHA-3, para minimizar el riesgo de colisiones.
- **Revisión de estándares y protocolos de seguridad:** Los estándares criptográficos deben actualizarse constantemente para seguir el ritmo de las investigaciones sobre seguridad, y así evitar el uso de algoritmos obsoletos que sean vulnerables a colisiones.

¿Qué es un algoritmo de hash?

Un algoritmo de hash es una función determinista que:

- Toma una entrada de cualquier tamaño (string, archivo, etc.)
- Y produce una salida de tamaño fijo (por ejemplo, 256 bits en SHA-256)
- De forma que:

- Es rápido de calcular
- Es prácticamente irreversible
- Y resiste colisiones (en teoría)

¿Cómo funciona por dentro (ejemplo: SHA-256)?

1. Preprocesamiento (Padding)

- Primero se convierte el mensaje a binario.
- Luego se le agrega relleno (padding) para que su longitud sea múltiplo de 512 bits.
- Se añade al final la longitud original del mensaje.

Esto es para que el algoritmo procese todo en bloques del mismo tamaño.

2. Inicialización de variables

- SHA-256 utiliza 8 variables iniciales de 32 bits cada una (se llaman A, B, C, D, E, F, G, H).
- También usa 64 constantes llamadas $K[i]$, predefinidas.

3. Proceso por bloques de 512 bits

- Cada bloque de 512 bits se divide en 64 "palabras" de 32 bits.
- Se hace una especie de mezcla compleja de operaciones en un bucle de 64 rondas:
 - Sumas binarias (mod 2^{32})
 - Rotaciones a la derecha (ROT)
 - Funciones lógicas (XOR, AND, NOT, etc.)
 - Se actualizan las variables A–H en cada ronda

4. Actualización del hash

- Al finalizar un bloque, se suman las variables A–H con los valores anteriores.
- Si hay más bloques, se repite el proceso usando los nuevos valores.

5. Resultado final

- El resultado es la concatenación de los valores A–H finales: un **digest de 256 bits** (64 caracteres hexadecimales).

Propiedades que debe cumplir:

- **Determinismo:** la misma entrada siempre da el mismo hash.
- **Avalancha:** un pequeño cambio genera un hash completamente distinto.
- **Resistencia a colisiones:** muy difícil encontrar dos entradas distintas con el mismo hash.
- **Resistencia a preimagen:** no se puede calcular la entrada a partir del hash.

4. ¿Para qué se usa un Hash?

Los hashes tienen una gran variedad de aplicaciones en tecnología, ya que ofrecen una forma eficiente de asegurar datos, garantizar la integridad de la información y proteger la privacidad. A continuación, detallo algunos de los usos más importantes de los hashes, explicando cómo y por qué se utilizan en diversas áreas:

Verificar si un archivo fue modificado

¿Cómo funciona?

Cuando descargas un archivo desde Internet, el sitio web a menudo te proporciona un valor hash (como un código alfanumérico) junto con el archivo. Este hash es el resultado de aplicar una función de hash a los datos del archivo original. Una vez que tienes el archivo en tu computadora, puedes calcular su hash localmente y compararlo con el hash proporcionado.

- **Si los hashes coinciden**, significa que el archivo no ha sido alterado.
- **Si los hashes no coinciden**, el archivo probablemente ha sido modificado o corrupto.

Ejemplo práctico:

Imagina que descargas un archivo de un software desde un servidor, como un sistema operativo o una aplicación. El servidor proporciona un hash como "d41d8cd98f00b204e9800998ecf8427e".

Luego, después de descargar el archivo, calculas su hash y obtienes "d41d8cd98f00b204e9800998ecf8427e" también, lo que te indica que el archivo no ha sido alterado.

- **Aplicaciones comunes:** Verificar la integridad de los archivos durante descargas, actualizaciones de software o copias de seguridad.

Guardar contraseñas de forma segura

¿Cómo funciona?

En lugar de almacenar contraseñas en texto claro (como "123456" o "password123"), las plataformas web almacenan el hash de la contraseña. Esto asegura que incluso si un atacante obtiene acceso a la base de datos de contraseñas, no podrá ver las contraseñas reales, sino solo los valores hash.

- **Proceso de hash de contraseñas:**

1. El usuario ingresa su contraseña (por ejemplo, "superSecreta123").

2. El sistema aplica una función hash (como SHA-256, bcrypt, o Argon2) sobre la contraseña, produciendo un hash único.
 3. En lugar de guardar "superSecreta123", se guarda el hash, que es algo como "a2c4f7b78f9f9bd13b7c3a75a12aeb5c".
- Cuando el usuario vuelve a ingresar su contraseña, el sistema vuelve a aplicar el hash y compara los resultados. Si los hashes coinciden, la contraseña es válida.

¿Por qué usar hashes para contraseñas?

El principal beneficio es la seguridad. Aunque un atacante tenga acceso a la base de datos de contraseñas, no podrá ver ni recuperar las contraseñas originales porque solo verá un conjunto de hashes. Además, muchas implementaciones añaden un "salt" (un valor aleatorio) al proceso de hash para hacer que cada hash sea único, incluso si dos personas tienen la misma contraseña.

- **Aplicaciones comunes:** Plataformas web (Redes sociales, correo electrónico, bancos en línea), sistemas operativos, aplicaciones móviles.

Evitar trampas en videojuegos

¿Cómo funciona?

En los videojuegos, especialmente en aquellos que tienen un componente en línea, el uso de hashes ayuda a garantizar la integridad de los archivos del juego y evitar trampas o modificaciones no autorizadas en los archivos del juego.

- **Archivos modificados:** Los jugadores pueden intentar modificar los archivos del juego para ganar ventajas (por ejemplo, aumentando la velocidad o desbloqueando elementos sin hacer progreso). Para evitar esto, el juego calcula un hash del archivo en su versión original y lo compara con los archivos locales del jugador.
- **Si el hash no coincide,** el sistema sabe que el archivo ha sido alterado y puede bloquear el acceso al servidor o mostrar un mensaje de advertencia.

Ejemplo práctico:

En un juego en línea multijugador, cuando un jugador conecta su cliente al servidor, el juego verifica que los archivos del cliente no hayan sido modificados con el fin de hacer trampas (como un "aimbot" o "wallhack"). Si el hash de los archivos del cliente no coincide con el hash esperado del servidor (que tiene la versión original de los archivos), el jugador será bloqueado o desconectado.

- **Aplicaciones comunes:** Juegos en línea (Fortnite, Call of Duty, etc.), aplicaciones de software que requieren un entorno de ejecución consistente.

Firmas digitales y autenticación de mensajes

¿Cómo funciona?

Las funciones de hash también se utilizan para crear firmas digitales, las cuales permiten verificar que un mensaje o documento proviene de una fuente confiable y no ha sido alterado.

- El **emisor** genera un hash del mensaje y lo cifra con su clave privada. Esto crea la firma digital.
- El **receptor** puede verificar la firma usando la clave pública del emisor. Si el hash del mensaje coincide con el hash cifrado de la firma digital, se confirma que el mensaje no ha sido modificado y proviene del emisor legítimo.

Ejemplo práctico:

Imagina que una empresa envía un contrato por correo electrónico. Para garantizar que el documento no haya sido alterado durante el envío, la empresa genera un hash del documento y lo firma digitalmente. El receptor puede usar la clave pública de la empresa para verificar que el contrato recibido coincide con el original y que no ha sido manipulado.

- **Aplicaciones comunes:** Comercio electrónico, transacciones bancarias, comunicación segura en redes.

Verificación de integridad en sistemas distribuidos y almacenamiento en la nube

¿Cómo funciona?

En sistemas distribuidos, donde los datos se almacenan en varios servidores o en la nube, los hashes son utilizados para verificar la integridad de los datos. Cada vez que se sube un archivo o se realiza una copia, se calcula un hash del archivo. Cuando el archivo es descargado o accedido nuevamente, se vuelve a calcular su hash para asegurarse de que no ha sido modificado durante el proceso de almacenamiento o transferencia.

- **Aplicaciones comunes:** Servicios de almacenamiento en la nube (Google Drive, Dropbox), sistemas de respaldo en servidores.

Optimización de bases de datos (Tablas Hash)

¿Cómo funciona?

Las funciones de hash también se utilizan en estructuras de tablas hash, que son una técnica de almacenamiento eficiente en bases de datos. En lugar de buscar datos en toda una lista (lo cual sería muy lento), las tablas hash usan una función de hash para mapear claves a ubicaciones específicas en memoria, lo que permite acceder rápidamente a los datos.

Ejemplo práctico:

En una base de datos de un sistema de gestión de usuarios, el nombre de usuario puede ser hashed para crear una clave única en la tabla, permitiendo que la búsqueda, inserción y eliminación de datos sea mucho más rápida que si tuviéramos que recorrer toda la base de datos.

- **Aplicaciones comunes:** Bases de datos de alta eficiencia, servidores de búsqueda y almacenamiento de datos.

¿Qué son las Tablas Hash?

Una tabla hash es una estructura de datos que permite almacenar y buscar información de manera muy rápida. Se usa para asociar claves (keys) con valores (values), y se basa en una función de hash para determinar dónde se guarda cada par clave-valor en la memoria.

¿Cómo funciona una tabla hash?

Las tablas hash utilizan una función de hash para convertir la clave en una ubicación única en la tabla (también conocida como "índice"). Este índice indica la posición en la memoria donde se almacenará el valor asociado con esa clave.

Pasos básicos en una tabla hash:

1. **Función de Hash:** Tienes una clave (por ejemplo, "nombre_usuario"). La función de hash toma esta clave y la convierte en un número (un índice). Por ejemplo, podría generar un número que corresponde a una posición en una lista o arreglo (en una tabla hash).
2. **Almacenamiento en la tabla:** El valor asociado con la clave se coloca en la ubicación determinada por la función de hash. Si la clave es "nombre_usuario" y la función de hash genera un índice de 5, el valor asociado con esa clave (como la contraseña o la dirección de correo) se guarda en la posición 5.
3. **Búsqueda eficiente:** Cuando quieras encontrar el valor de una clave, aplicas la misma función de hash a la clave. Esto te da el índice exacto donde está almacenado el valor. Si hay colisiones (cuando dos claves diferentes generan el

mismo índice), se usan técnicas adicionales para gestionarlas.

Ejemplo de uso de una tabla hash

Imagina que tienes un directorio telefónico y deseas encontrar el número de teléfono de alguien por su nombre. En lugar de buscar en toda la lista, usas una función de hash para convertir el nombre de la persona en un índice de una tabla, y ese índice te lleva directamente al número de teléfono asociado.

Ejemplo práctico: Supongamos que tienes las siguientes claves y valores en un directorio telefónico:

- **Clave:** "Juan", **Valor:** "123456789"
- **Clave:** "Ana", **Valor:** "987654321"

La función de hash convierte "Juan" y "Ana" en índices en la tabla, por ejemplo:

- "Juan" → Índice 3
- "Ana" → Índice 5

Y en esos índices, los valores "123456789" y "987654321" se almacenan respectivamente.

Cuando buscas el número de Juan, simplemente aplicas la función de hash a "Juan", lo que te lleva al índice 3, donde está almacenado su número.

Usos comunes de las tablas hash

1. **Bases de datos:** Las tablas hash se utilizan para indexar rápidamente datos y mejorar las consultas.
2. **Caches:** Las tablas hash se usan en sistemas de memoria caché para almacenar resultados de operaciones y evitar cálculos repetidos.
3. **Sistemas de autenticación:** Se utilizan para verificar contraseñas, como en el caso de las bases de datos de contraseñas, donde se almacena el hash de las contraseñas en lugar de las contraseñas en texto claro.
4. **Redes P2P (Peer-to-Peer):** Las tablas hash se emplean en sistemas distribuidos, como los usados por redes P2P para localizar y almacenar archivos.

5. ¿Cómo se usan los hashes en criptomonedas?

Las funciones de hash son fundamentales para que las criptomonedas funcionen. En especial en Bitcoin, los hashes actúan como el “pegamento” que une y protege toda la red.

Sin ellos, no existiría ni la blockchain, ni la minería, ni la seguridad que hace que nadie pueda hacer trampa.

Gracias a los hashes, cada bloque de datos queda conectado con el anterior, y se puede verificar que nadie haya modificado nada.

Además, permiten a los mineros competir por validar bloques y mantener viva la red.

A) Seguridad de los bloques – la estructura de la blockchain

¿Qué es un bloque?

Un bloque es como una "página" del libro de contabilidad de Bitcoin. Contiene:

- Una lista de transacciones
- La fecha/hora
- Un número (nonce)
- El hash del bloque anterior

¿Qué papel juega el hash?

Cada bloque tiene un hash propio que se calcula usando todo el contenido del bloque. Ese hash sirve como identificador único del bloque.

Y el siguiente bloque incluye ese hash dentro de su contenido.

Esto crea una cadena de bloques (block-chain).

¿Qué pasa si alguien intenta modificar algo?

Si un atacante cambia algo en un bloque anterior (por ejemplo, modifica una transacción), el hash de ese bloque cambia. Como el siguiente bloque depende de ese hash, toda la cadena se rompe desde ahí.

Por eso se dice que el hash garantiza la integridad de la cadena. Cuanto más larga la cadena, más imposible es modificar algo sin que se note.

B) Minería – la prueba de trabajo (Proof of Work)

¿Qué hacen los mineros?

Los mineros compiten por ser los primeros en encontrar un hash que cumpla una condición especial: que el hash comience con cierta cantidad de ceros (por ejemplo, 19 ceros).

Esto no se puede calcular directamente. Lo único que pueden hacer es probar millones de combinaciones diferentes de datos (principalmente cambiando el número llamado nonce) hasta que uno de esos intentos da un hash válido.

Es como un juego de adivinanza con millones de dados.

¿Qué se hashéa?

Los mineros calculan el hash de:

Hash del bloque anterior + datos de las transacciones + marca de tiempo + nonce

Prueban diferentes nonce hasta que encuentran un hash que empiece con muchos ceros.

Ejemplo:

Supongamos que se quiere encontrar un hash que empiece con 0000.

Intento: nonce = 582304 → Hash: 12ad2f8c...

Intento: nonce = 582305 → Hash: b92b12c7...

Intento: nonce = 582306 → Hash: 00007d9a... **¡Éxito!**

El primero en lograrlo gana la recompensa en bitcoins (hoy son 3.125 BTC por bloque, tras el último halving en abril 2024).

¿Por qué esta dificultad?

Porque si fuera fácil encontrar un hash válido, la red sería vulnerable a ataques (cualquiera podría agregar bloques falsos).

La dificultad se ajusta automáticamente cada 2016 bloques (aprox. cada 2 semanas), para que siempre se tarde unos 10 minutos en promedio en minar un bloque nuevo.

Curiosidades y detalles técnicos

La prueba de trabajo es costosa a propósito

- El algoritmo de hash que usa Bitcoin es SHA-256 (Secure Hash Algorithm 256 bits).
- No es lento, pero como se repite miles de millones de veces por segundo, la minería requiere mucha potencia.
- Esto ha llevado a la creación de hardware especializado (ASICs) que solo hacen SHA-256 a toda velocidad.

El hash como “pegamento”

- El hash del bloque anterior es lo que conecta los bloques.
- Esto significa que la cadena está asegurada matemáticamente.
- Cuantos más bloques se agregan después del tuyo, más difícil es modificarlo (porque tendrías que rehacer todos los hashes posteriores).

Los hashes también protegen las transacciones

Cada transacción tiene su propio hash (TxID). Esto sirve para:

- Identificarla de forma única.
- Verificar su integridad.
- Asegurarse de que no se duplicó ni se modificó.

¿Y los famosos “halvings”?

- Cada 210,000 bloques (aproximadamente cada 4 años), la recompensa de minería se reduce a la mitad.
- En 2009 eran 50 BTC por bloque.
- En 2024 bajó a 3.125 BTC.
- Esto hace que minar sea más difícil, y que el valor de Bitcoin tienda a subir por escasez controlada.

¿Sabías que...?

- El bloque 0 (el primero) se llama Genesis Block y fue creado por Satoshi Nakamoto en enero de 2009.

Su hash es:

000000000019d6689c085ae165831e93 . . .

- El hash de un bloque siempre se empieza a calcular sin saber si va a funcionar. Los mineros solo lo saben cuando lo prueban.

¿Es legal la minería de criptomonedas?

Sí, en la mayoría de los países es legal, pero depende del país.

- **Legal:** En muchos países como España, México, EE. UU., Alemania, Argentina, etc., minar criptomonedas es totalmente legal
- **Regulada o prohibida:** En algunos países como China o Marruecos, la minería está limitada o prohibida, sobre todo por razones de:
 - Consumo eléctrico
 - Control del dinero (los gobiernos quieren controlar la economía)
 - Riesgo de fraude

¿Qué es la minería realmente?

La minería no es hackeo, es como ser un “contador digital” para la red.

Imagina que hay millones de transacciones de Bitcoin dando vueltas.

Los mineros:

- Juntan esas transacciones en un “bloque”
- Se esfuerzan por encontrar un hash válido (como te conté antes)
- Si lo logran, validan ese bloque
- Y la red lo acepta como oficial

A cambio, la red les paga con criptomonedas nuevas (recompensa) + comisiones de transacciones.

Es como ser recompensado por hacer el trabajo de verificar que todo está bien.

Entonces... ¿los mineros son hackers? NO. Todo lo contrario.

Los mineros:

- Siguen las reglas del sistema (el protocolo de Bitcoin o Ethereum)
- Ayudan a que la red sea segura
- No roban nada, no espían, no hackean

De hecho, sin mineros no habría criptomonedas. Ellos son los que hacen que la red siga funcionando.

Ahora, sí hay hackers en el mundo cripto, como en todo sistema:

- Estafas
- Robos de claves privadas
- Malware que mina en tu compu sin permiso

Pero eso no tiene nada que ver con la minería honesta y legal.

¿Y cómo se gana dinero?

A. Minería

- Comprás hardware (muy caro hoy en día).
- Lo conectás a internet, configurás el software.
- Pagás electricidad.
- Si encontrás un bloque o participás en un grupo (pool), ganás una parte de los BTC.

Hoy en día, la minería casera no es tan rentable en la mayoría de países por el alto costo eléctrico. Por eso muchos usan “minería en la nube” o se unen a pools (grupos de mineros).

B. Trading (compra/venta)

- Comprar barato y vender caro (como en la bolsa).
- Se puede hacer legalmente en exchanges (plataformas) como Binance, Coinbase, etc.

C. Staking (en criptos como Ethereum)

- Dejá tus criptos “bloqueadas” como garantía y ganás recompensas.
- No necesitás hardware caro ni gastar electricidad.

¡Cuidado!

- Aunque la minería es legal, declarar ingresos es obligatorio en muchos países.
- Hay muchas estafas disfrazadas de minería, como páginas que te prometen que ganarás miles sin hacer nada.

6. ¿Qué es una Rainbow Table?

Una Rainbow Table (tabla arcoíris) es una técnica que los atacantes usan para descifrar contraseñas o cualquier otro dato que haya sido cifrado mediante un hash. Su propósito es romper la seguridad de las funciones hash, facilitando el proceso de obtener las contraseñas originales a partir de sus hashes. Aquí te explico en más detalle cómo funciona y por qué es un problema de seguridad.

¿Cómo funciona una Rainbow Table?

1. Precalcular pares de datos:

La Rainbow Table es esencialmente una lista muy grande de entradas y sus correspondientes valores de hash.

- **Entradas:** Podrían ser contraseñas comunes, combinaciones simples, frases populares, etc.
- **Hashes:** Son el resultado de aplicar una función hash (como MD5, SHA-1, SHA-256) a esas entradas.

2. Ejemplo: Si alguien utiliza una contraseña simple como "12345", la tabla contendría un par como este:

- Entrada: "12345"
- Hash: "a2c4f7b78f9f9bd13b7c3a75a12aeb5c" (un ejemplo de hash generado por MD5)

3. Buscar coincidencias:

Cuando un atacante obtiene un hash (por ejemplo, el hash de una contraseña almacenada en una base de datos), puede buscar ese hash en la Rainbow Table. Si encuentra una coincidencia, el atacante sabe exactamente cuál es la entrada original (la contraseña en este caso).

4. Ventaja de las Rainbow Tables:

En lugar de calcular los hashes una y otra vez para cada posible entrada durante un ataque (lo cual sería muy lento), la Rainbow Table permite al atacante ahorrar tiempo, ya que las combinaciones ya han sido precalculadas. Esto acelera mucho el proceso de descifrar contraseñas.

Limitaciones de las Rainbow Tables

Aunque las Rainbow Tables son poderosas, tienen algunas limitaciones que las hacen menos eficaces en sistemas bien diseñados:

1. Requieren mucho espacio de almacenamiento:

Cuanto más grande sea la tabla, más contraseñas puede contener. Esto significa

que las tablas pueden llegar a ser muy grandes. Por ejemplo, una tabla que contenga contraseñas de 8 caracteres con caracteres alfanuméricos puede tener un tamaño de varios terabytes.

2. **No funcionan con "salts":**

El mayor contramedida contra las Rainbow Tables es el uso de salts. Un "salt" es un valor aleatorio que se agrega a la contraseña antes de hacerle el hash.

Esto significa que incluso si dos personas tienen la misma contraseña, sus hashes serán diferentes porque cada una tiene un salt único. Por lo tanto, las Rainbow Tables no pueden ser utilizadas para atacar contraseñas que usan salts, ya que el hash final será distinto cada vez.

¿Qué son los "salts" y cómo previenen las Rainbow Tables?

Los salts son valores aleatorios añadidos a la contraseña antes de aplicarle el hash. Este proceso crea una diferencia significativa en los hashes generados, haciendo inútiles las Rainbow Tables.

Por ejemplo, si tienes la contraseña "12345" y le añades un salt como "abc123", el hash generado será completamente diferente que si solo hashéaras "12345".

Ejemplo:

- Contraseña: "12345"
- Salt: "abc123"
- Hash generado: sha256("12345abc123")

En este caso, aunque alguien tenga la Rainbow Table con hashes precalculados para "12345", no podrá usarla, porque el salt cambia completamente el valor final del hash.

Ventajas del uso de salts:

1. **Protege contra ataques de Rainbow Tables:** Hace que las tablas precalculadas sean inútiles.
2. **Evita ataques de diccionario:** Incluso si alguien intenta probar una lista de contraseñas comunes, los salts garantizan que los hashes no sean iguales.

Uso de Rainbow Tables en la práctica

Las Rainbow Tables se utilizan típicamente en ataques de fuerza bruta o ataques de diccionario cuando los atacantes obtienen acceso a bases de datos de contraseñas hashadas. Estas tablas también se pueden usar en aplicaciones de hacking ético para pruebas de penetración, aunque en un contexto controlado y ético.

Ejemplo de ataque:

1. Un atacante obtiene una lista de hashes de contraseñas de una base de datos comprometida.
2. Usa una Rainbow Table para encontrar los hashes que coinciden con contraseñas comunes.
3. Si el sistema no usa salts, el atacante rápidamente puede obtener las contraseñas originales.

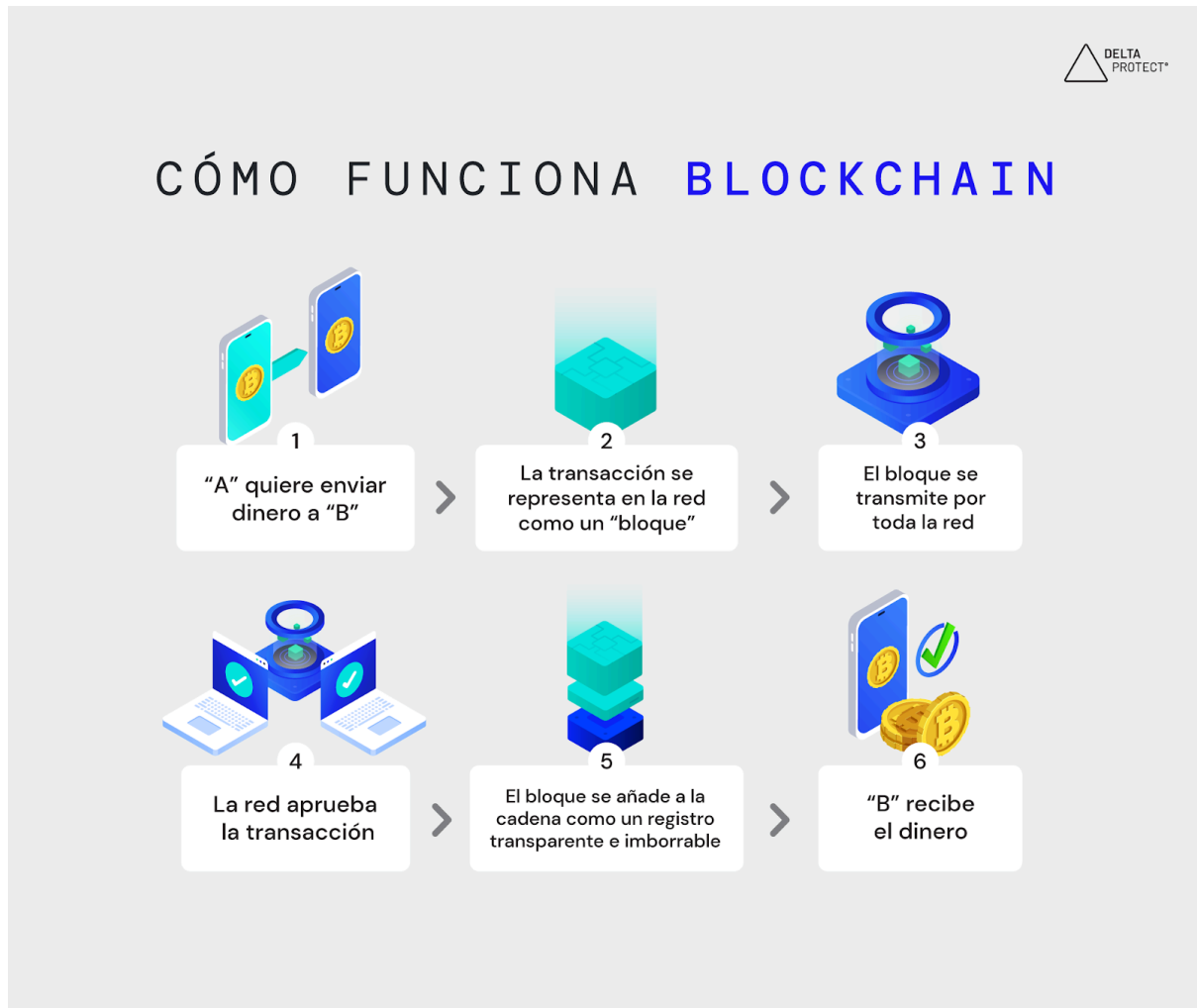
¿Cómo prevenir los ataques de Rainbow Tables?

- **Usar Salts:** Siempre agregar un valor único aleatorio antes de hashear las contraseñas.
- **Funciones de hash lentas:** Utilizar algoritmos como bcrypt, scrypt o Argon2, que están diseñados para ser lentamente computacionales, lo que hace mucho más difícil realizar ataques masivos, incluso sin Rainbow Tables.
- **Contraseñas fuertes:** Asegurarse de que las contraseñas sean largas, complejas y únicas para cada usuario.
- **Autenticación de dos factores (2FA):** Agregar una capa extra de seguridad que proteja las contraseñas incluso si son descifradas.

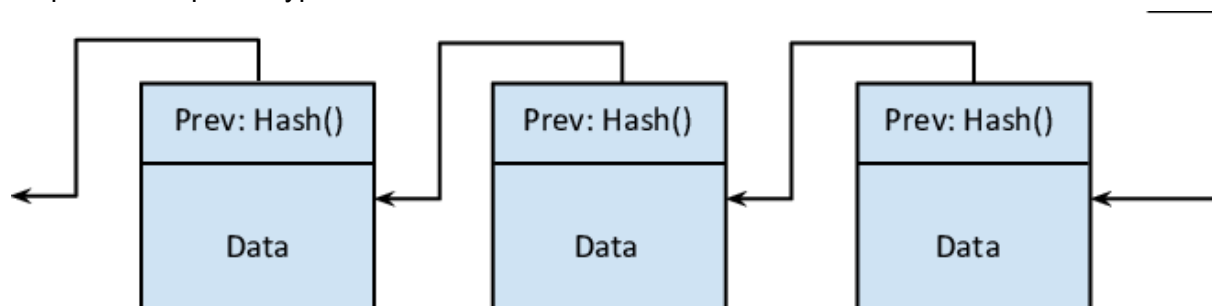
7. Esquemas

Aquí podemos visualizar diversos esquemas sobre algunos conceptos anteriormente explicados:

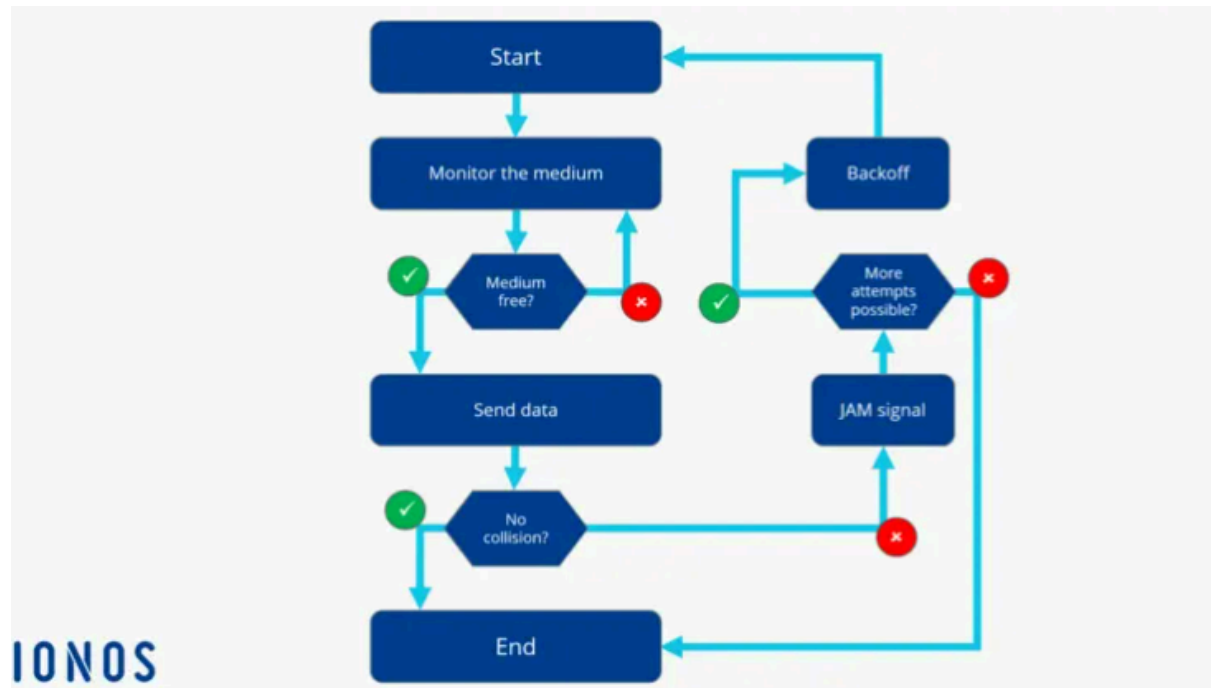
Esquema Funcionamiento Cryptos:



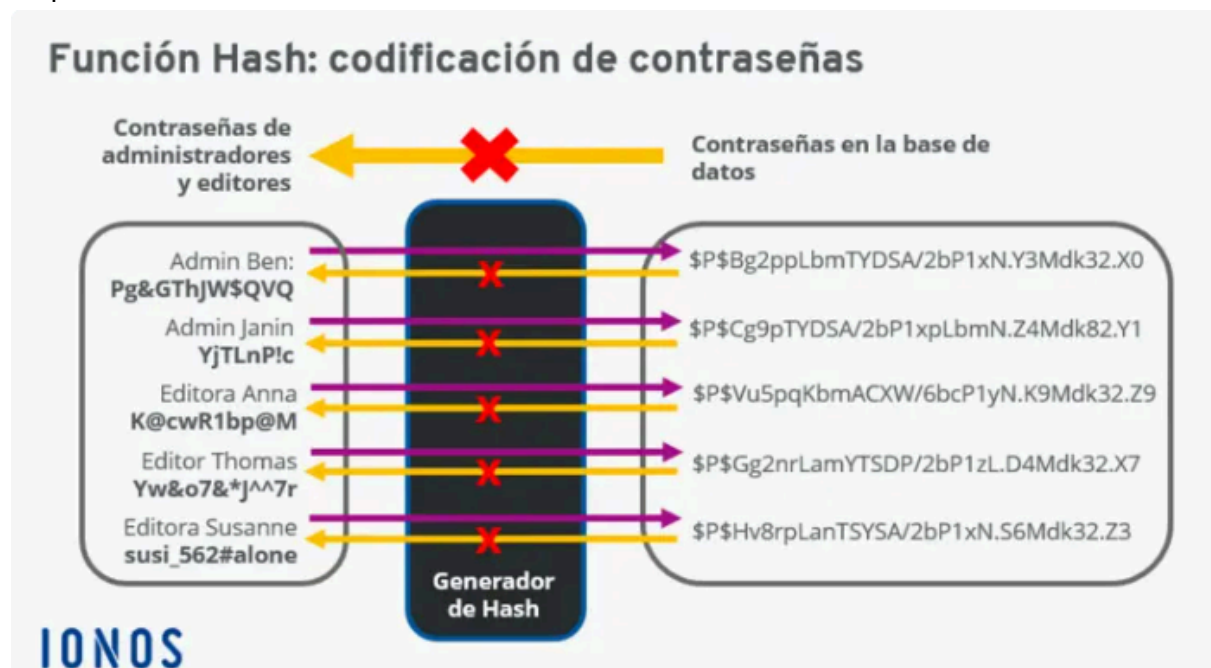
Esquema Bloques Cryptos:



Esquema de Colisión:



Esquema de Funcionamiento del Hash:



Esquema Salt:



8. Videos

En este apartado dejamos diversos enlaces que nos llevan a visualizar diversos conceptos anteriormente vistos para entenderlos gráficamente mejor.

Video Rainbow Tables:

<https://www.youtube.com/watch?v=QEqRpoP1K0s>

Video explicando hash:

<https://www.youtube.com/watch?v=2BldESGZKB8>

Salt y securización de la Base de datos:

<https://www.youtube.com/watch?v=zt8Cocdy15c&t=97s>

Hash en cryptos:

<https://www.youtube.com/watch?v=mMxkxwPSfvo>

Funcionamiento en cryptos:

<https://www.youtube.com/watch?v=FmtY2hmg04>